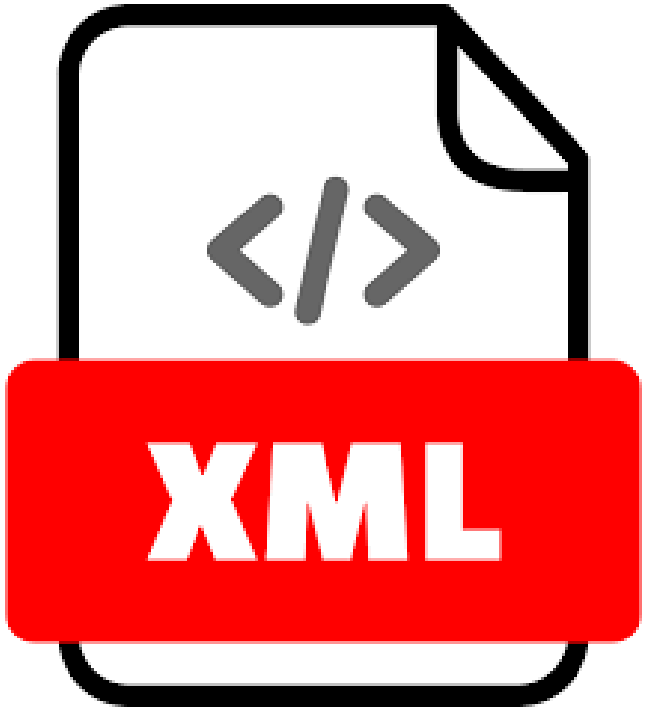
XML Basics and Structured Data Integration

XML Basics and DTD

Outline

- XML structure, syntax, DTD/Schema basics
- Reading and writing XML in PHP
- Using XML for configurations or data exchange
- Introduction to JSON as a modern alternative (optional)
- Quick discussion of PHP as OOP: classes, objects, inheritance (optional)



What is XML?

XML (eXtensible Markup Language) is a markup language used to store and transport structured data. It is a language designed to describe the data.

Key Characteristics:

- Human-readable and machine-readable
- Uses custom tags
- Platform independent
- Commonly used for data exchange between systems

Example Uses:

- Configuration files
- Data exchange between applications
- Web services

XML

```
<firstName>Maria</firstName>
<lastName>Roberts</lastName>
<dateBirth>12-11-1942</dateBirth>
```

HTML

```
<font size="3">Maria Roberts</font>
<b>12-11-1942</b>
```

Basic XML Structure

XML documents follow a hierarchical structure.

Example:

```
<students>
  <student>
    <name>Maria</name>
    <course>IT</course>
  </student>
</students>
```

Rules:

- Must have a single root element
- Tags must be properly nested
- Tags are case sensitive

XML Document: Serialized Representation

Important XML Syntax Rules:

- Every opening tag must have a closing tag
- Tags must be properly nested
- Attribute values must be quoted
- XML is case sensitive

```
<book>
  <authors>
    <author id="47">John Doe</author>
    <author id="58">Peter Pan</author>
  </authors>
  <title>Database systems</title>
  <price>29</price>
  <keywords>
    <keyword>SQL</keyword>
    <keyword>relational</keyword>
  </keywords>
</book>
```

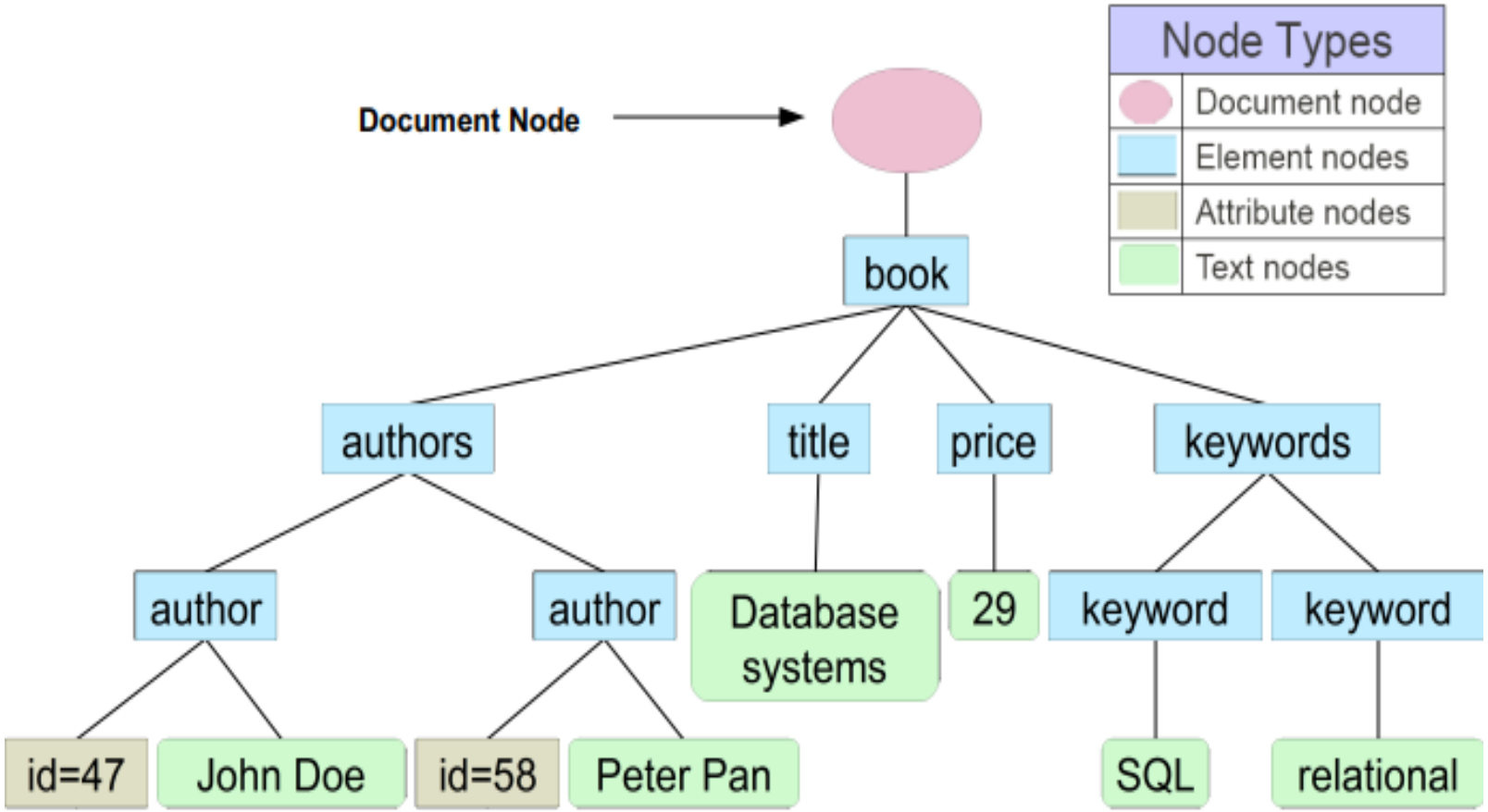
Root element

Attribute

Element

Text node (Data)

XML Document: Parsed-Hierarchical



Well-Formed vs. Valid XML Documents

A well-formed XML document is a document that follows basic rules:

1. It must have one and only one root element
2. Each element begins with a start tag and ends with an end tag
3. An element can contain other elements, attributes, or text nodes
4. Attribute values must be enclosed in double quotes. Text nodes, on the other hand, should not.

A valid XML document is BOTH:

1. A well-formed XML document
2. A document compliant with the rules defined in an XML schema or a Document Type Definition (DTD) document.

Example

```
<?xml version="1.0"?>
<BusinessCard>
  <Name>Joe Marini</Name>
  <phone type="mobile">(415) 555-4567</phone>
  <phone type="work">(800) 555-9876</phone>
  <phone type="fax">(510) 555-1234</phone>
  <phone type="home">(425) 555-8989</phone>
  <email>joe@joe.com</email>
</BusinessCard>
```

Why XML is still relevant?

- XML is excellent for system-to-system communication, especially when systems are built using different technologies.
- Languages like PHP, Java, and Python all have built-in libraries for reading and writing XML.
- XML is better when dealing with highly structured or hierarchical data. It clearly represents nested relationships, which is helpful for large structured documents.
- While JSON is lighter and more popular for modern APIs, XML remains important for structured data validation, legacy systems, and complex data exchange.

DTD Basics

Applying rules to XML Document

DTD Basics

DTD (Document Type Definition) defines the structure of an XML document

It specifies:

- Allowed elements
- Allowed attributes
- Element hierarchy

Example:

```
<!DOCTYPE note [
 <!ELEMENT note (to,from,message)>
 <!ELEMENT to (#PCDATA)>
 <!ELEMENT from (#PCDATA)>
 <!ELEMENT message (#PCDATA)>
]>
```

Format of a DTD

- A Document Type Declaration has the following format:

```
<!DOCTYPE Name [
    Document Type Definition
]>
```

Where *Name* is the name of the root node of the document and *the Document Type Definition* defines the content rules.

Declaring Elements

- Element declarations have the format

```
<!ELEMENT Name ContentSpecification>
```

Name is the name of the element

ContentSpecification defines the valid contents for the elements

- Elements can contain:
  - EMPTY content – element has no children
  - ANY content – element has no content constraints
  - Element content – element can contain the listed child elements, but that's it
  - Mixed content – element can contain elements and character data

Declaring Elements - Example

- Element with ANY content:

```
<!ELEMENT MISCELLANEOUS ANY>
```

- Empty Element:

```
<!ELEMENT IMG EMPTY>
```

- Character Data Content:

```
<!ELEMENT NAME (#PCDATA)>
```

- Element Content (Sequence):

```
<!ELEMENT CD (TITLE, ARTIST)>
```

- Element Content (Choice):

```
<!ELEMENT DWELLING (HOUS |APT | TRAILER)
```

Declaring Elements - Example

- Mixed Content:

```
<!ELEMENT TITLE (#PCDATA | SUBTITLE) >
```

- Specifying element population
  - To modify the element declaration to indicate how many elements are allowed, you can use the ?, +, * characters
  - ? means zero or one of the indicated element
  - + means one or more of the indicated element
  - * means zero or more the indicated element
- Example:

```
<!ELEMENT LETTER (A?, B*, C+)
```

- Means LETTER can contain 0 or 1 A, followed by 0 or more B, followed by one or more C elements

Declaring Attributes

- All attributes intended for use in a document must be declared via an attribute list declaration
- An attribute list declaration has the format:

```
<!ATTLIST ElemName AttrName Type DefaultDecl>
```

- Where:
  - *ElemName* is the name of the element where the attribute can be used
  - *AttrName* is the name of the attribute
  - *Type* id the attribute's type
  - *DefaultDecl* is the default declaration

Declaring Attributes

- Attribute can be
  - String type, indicated by the CDATA keyword. This means the attribute can be any literal string
  - Enumerated type, which means the attribute can be one of several potential values
  - Tokenized type
- String Type

```
<!ATTLIST PACKAGE status CDATA “OK”>
```

- Enumerated Type:

```
<!ATTLIST DWELLING type (house | apt | trailer)>
```

Declaring Attributes

- The Default Declaration
  - #REQUIRED

```
<!ATTLIST PACKAGE ontime (yes|no) #REQUIRED>
```

  - #IMPLIED

```
<!ATTLIST PACKAGE ontime (yes|no) #IMPLIED>
```

  - Default Value

```
<!ATTLIST PACKAGE ontime (yes|no) “yes”>
```

  - #FIXED

```
<!ATTLIST PACKAGE ontime CDATA #FIXED “yes”>
```

EXAMPLE – DTD

```
<?xml version="1.0"?>

<!DOCTYPE BusinessCard [
   <!ELEMENT BusinessCard (Name, phone+, email?)>
   <!ELEMENT Name(#PCDATA)>
   <!ELEMENT phone(#PCDATA)>
   <!ATTLIST phone type(mobile|fax|work|home) #REQUIRED>
   <!ELEMENT email (#PCDATA)>
]>

<BusinessCard>
  <Name>Joe Marini</Name>
  <phone type="mobile">(415) 555-4567</phone>
  <phone type="work">(800) 555-9876</phone>
  <phone type="fax">(510) 555-1234</phone>
  <phone type="home">(425) 555-8989</phone>
  <email>joe@joe.com</email>
</BusinessCard>
```

XML Schema Basics

XML Schema (XSD) is a more powerful alternative to DTD.

Advantages:

- Written in XML
- Supports data types
- More flexible validation

Example:

```
<xs:element name="age" type="xs:integer"/>
```

XML Schema helps validate the correctness of XML data.

Reading XML Data Using PHP

XML File (Saved as xmldata.xml)

```
<?xml version="1.0" encoding="UTF-8" ?>
<employees>
     <employee>
         <id>1</id>
         <firstName>Leonardo</firstName>
         <lastName>DiCaprio</lastName>
         <age>51</age>
     </employee>
     <employee>
         <id>2</id>
         <firstName>Johnny</firstName>
         <lastName>Depp</lastName>
         <age>62</age>
     </employee>
     <employee>
         <id>3</id>
         <firstName>Andrew</firstName>
         <lastName>Garfield</lastName>
         <age>42</age>
     </employee>
</employees>
```

Using PHP to read XML data

```
simplexml_load_file('xml file');
```

- The **simplexml_load_file()** function converts an XML document to an object.
- Data can be accessed like object properties

PHP Code

```
<?php
  $xml = simplexml_load_file('xmldata.xml');

  foreach($xml->employee as $employee){
    echo "ID No: ".$employee->id."<br/>";
    echo "First Name: ". $employee->firstName."<br/>";
    echo "Last Name: ". $employee->lastName."<br/>";
    echo "Age : ". $employee->age."<br/>";
    echo "<hr/>";
  }

?>
```

Element in XML to start getting the data

Gets the content of each node

Adding XML Element using PHP

```
<?php
   if(isset($_POST['insert'])){

      if(file_exists("xmldata.xml")){
         $employees = simplexml_load_file('xmldata.xml');
         $employee = $employees->addChild('employee');
         $employee->addChild('id', $_POST['id']);
         $employee->addChild('firstname', $_POST['fname']);
         $employee->addChild('lastname', $_POST['lname']);
         $employee->addChild('age', $_POST['age']);
         file_put_contents('xmldata.xml', $employees->asXML());
         header('location:formXML.php');
      }
   }
?>
```

reads the XML file and converts it into an object using SimpleXML. It becomes easy to manipulate in PHP

represents the root node <employees>, and append the <employee> element

writes that content back into the file.

converts the XML object back into XML text format.

Using XML for Configuration

XML is often used for application configuration.

Example (config.xml):

```
<config>
  <database>
     <host>localhost</host>
     <user>root</user>
  </database>
</config>
```

Applications read this file to load settings.

XML for Data Exchange

XML allows different systems to exchange structured data.

Example Scenario:

- A website sends order data to another system
- XML acts as a standard format

Example:

```
<order>
   <id>1001</id>
   <customer>John</customer>
</order>
```

Introduction to JSON

Manipulation of Data

Introduction to JSON

- **JSON (JavaScript Object Notation)** is a lightweight data format and is a syntax for storing and exchanging data.
- It is a text-based format and can easily be sent to and from a server.
- JSON syntax is derived from JavaScript object notation syntax:
  - Data is in name/value pairs
  - Data is separated by commas
  - Curly braces hold objects
  - Square brackets hold arrays

JSON Data – A Name and a Value

- JSON data is written as **name/value pairs**.
- A name/value pair consists of a field name (in a double quotes) followed by a colon, followed by a value:
  - Example: “name”: “Facifica”
  - JSON Object : { “name”:”Facifica”}

JSON in PHP Example

- PHP can encode and decode JSON.
- The **json_encode()** function is used to encode a value to JSON format

```
<?php
$actor_age = array("Leonardo DiCaprio" => "51", "Johnny Depp" =>
"62", "Andrew Garfield" => "42");
$json_data = json_encode($actor_age);

echo $json_data;
?>
```

- **Purpose:** Easier way to store and transport data. JSON data can be easily stored in databases, files, API responses, or local storage in browsers.

JSON in PHP Example

- The **json_decode()** function is used decode a JSON object into a PHP object or an associative array

```
<?php
$json_data = '{"Leonardo DiCaprio":"51", "Johnny Depp":"62",
"Andrew Garfield":"42"}';

$object_data = json_decode($json_data);

foreach($object_data as $name => $age){
    echo $name . " : " . $age . "<br>";
}
?>
```

PHP as OOP

Good to know

Introduction to PHP OOP

- **Object-Oriented Programming (OOP)** organizes code into objects. It is a coding methodology.
- Makes code more modular and reusable.
- Not always the best approach to every single project.
- Key Concepts:
  - **Class** – blueprint of an object
  - **Object** – instance of a class
  - **Properties** – variables inside a class
  - **Methods** – functions inside a class

PHP OOP Example

```
<?php
class ActorList {                          //class definition
   private $actors = array(                //private property of a class
        "Leonardo DiCaprio" => 51,
        "Johnny Depp" => 62,
        "Andrew Garfield" => 42);

   public function displayActors(){     //method
        foreach($this->actors as $name => $age){
           echo $name . " : " . $age . "<br>";
        }
   }
}

$actors = new ActorList();            //creates an instance of a class
$actors->displayActors();             //calling the method

?>
```

Inheritance in PHP

Inheritance allows a class to reuse another class.

```
<?php
class Person {   // Parent Class
 protected $actors = array("Leonardo DiCaprio" => 51, "Johnny Depp" => 62,
"Andrew Garfield" => 42);
}

class ActorList extends Person {        // Child Class
    public function displayActors(){
          foreach($this->actors as $name => $age){
                echo $name . " : " . $age . "<br>";
          }
     }
}
$actors = new ActorList();             // Create object
$actors->displayActors();              // Call method
?>
```

XML Basics and Structured Data Integration

XML Basics and DTD